

Web Site Project: The Internet History in Canada

Abstract:

This website includes two topics of technology, legislation and a page for presenting the statistics about the Canada's Internet in charts. Technology and legislation topics are gathered by research, and the collected data is saved in the form of the JSON files. Statistics will be collected by the server, as a proxy, from the Statcan API. The latest data will be extracted from the Statcan API and loaded in the memory to be ready to provide to clients. The front-end will have access to the technology, legislation and statistics data via the provided REST API. Front-end of the website provides a form for users to contribute content to the site. The form will contact the server by provided REST API to upload data including the possible pictures.

Planning:

The project life cycle was built based on Kanban method. The backlog started with following tasks:

- 1- Investigate and data gathering (Duration: 16 hours)
- 2- Create Wireframe (Duration: hrs.)
- 3- Develop and backend and design REST API (Duration: 40 hours)
- 4- Front-end coding for getting and posting data (Duration: 40 hours)
- 5- Design HTMLs using a templating engine to dynamically render final pages (Duration: 40 hours)
- 6- Layout and styling (Duration: 32 hours)
- 7- Deploying the website on Render (Duration: 1 hour)
- 8- HTML validation and accessibility check (Duration 2 hours)

Investigation:

For this task, I used the available online sources to collect data about different technologies and the time they were introduced in Canada. I also found some sophisticated APIs on Statistics Canada, called Statcan, related to the Internet. Therefore, I extended the time allocated for this task to study the structure of the data provided by these APIs.

Each table in Statcan is called cube with a specific PID number. This is a multidimensional table, and clients use coordinates in body of API requests to receive the intended data dimensions [1].

Wireframe:

Wireframe was created in Figma and shared its prototype link with a few people for review via Google Forms:

Figma:

<https://www.figma.com/proto/o7SXnsf0WmcbUzvriYqwVi/Untitled?node-id=85-2553&p=f&t=ULrLgc60bsftUWqw-0&scaling=min-zoom&content-scaling=fixed&page-id=0%3A1&starting-point-node-id=64%3A2240&show-prot-sidebar=1>

Google Forms:

https://docs.google.com/forms/d/e/1FAIpQLSe-6YZG0YfqtuRj8f7WqOG7zmbj_ZQK7JGOUw--tX5C9TuvKA/viewform?usp=dialog

The responses I received is shown in image 1, and I addressed them during the site development:

Timestamp	Is the site easy to navigate?	Did you get confused at any point?	How would you rate the layout of topics?	How would you rate the navigation tools?	What would you change or add to the site?
2/24/2026 8:56:5	4	No		4	3 The structure is overall ok. I recommend following improvements: 1- fix the menu and logo on top of the page. 2-Make the menu items shorter. For example, As a UI/UX designer, I think the wireframe looks good overall. To make it better, try not to use bold fonts. Also, it'd be helpful to add a scroll-to-top button on pages with lots of content so users can quickly jump back to the header.
2/24/2026 16:39:	5	No		5	4 The interface is easy to navigate. The main improvement would be making the top menu fixed (sticky). also, consider adding a Contact page.
2/24/2026 16:59:	5	No		4	5

Image1- The result of assessments in Google Forms

Backend and REST API:

In this task, I designed a server as a proxy for Statcan. At early stage of the design, the server would receive request from the client and send the according API request to Statcan and then send the received data to the client. In the testing stage of this task, and when I started to design template engines and layouts, I

noticed that it causes significant delays. In addition, sometimes Statcan was unavailable, and would make my web site un operational too.

To mitigate this issue, I redesigned the backend, so the server fetches all the required data from Statcan when it starts, and before being available for clients. This approach eliminates the delays for fetching data from Statcan. It also enables the server to use the latest fetched data from Statcan if Statcan is not available. It can prevent potential issues in depolying the site, especially for testing and assessment.

To speed up the process of fetching data from the Statcan, I applied Promise.all(), so it helps to send several requests to the Statcan at the same time instead of using await keyword, which would wait for each request to complete before sending the next one. However, I noticed that sometimes Statcan gets overwhelmed and send back a response with status 429, which is an indication of too many requests [2]. To mitigate it I created a stopper function [3], so to send failed request again after 5 seconds.

Another feature in the server is the REST API for posting the content contributions to the server. The body of the request for this end point is JSON type, and it has a property that itself is an object and contains images in the format of base64. When data receives by this end point, the images will be converted to the binary and saved in the local storage [9]. The images get deleted from data and instead their addresses in the local storage will be recorded in the data. After this the JSON structire will be cleaned of unnecessary properties if they are empty and added to the corresponding JSON file in the database.

Front-end API coding:

At front-end, the main JavaScript code is in app.js. It will create instances of multiple classes that are designed for fetching data from the server, rendering HTMLs and creating charts. It starts with fetching data from the server, initializing template engine instances and rendering HTMLs for each page, create charts and timeline instances.

For the technology and legislation page, I used vis.js timeline to illustrate the timeline of the Internet events [4][5][6]. The event listeners on the timeline items enables app to scroll to the corresponding content when items are clicked. To

achieve this feature, I used HTML data-* attribute to link element div of each history item to the items in the vis-timeline[7].

For the statistics data from Statcan, I used charts.js [8]. I created a LineChart class to use it for line charts. It takes the data and generated the chart based on the provided parameters and data. The LineChart is also a parent for other type of chart classes, which are DoughnutChart, Polarchart, and Barchart. The updateDataset() function for these child classes are overwritten and customized so they provide the desired output.

Template Engines:

The HTML of this project is based on the main HTML file called index. The body index has three elements, which are header, main and div.

The content in the main element of this HTML will be replaced with the rendered HTMLs which are provided by template engines based on the opened page. Template engines take the relevant data which are already fetched via APIs from the server and replace data with the placeholders. The base code of template engines is inspired from the week 11 and week 12 of the module CM1040. The challenge for the history_template_engine was that there were some nested arrays in the data and regular expressions could not identify which `{{/each}}` is for which loop in nested loops. To address it, I made the template engine to loop over each history item in the data, embeds a fragment html template for each item, and then replace the place holders of that fragment with the elements of that item.

The element div in the body is reserved for a dialog form. The content of div will be replaced with the coontribute_dialog.html, when user navigates to the technology of legislation page and it provides tool to users so they can contribute to the content of the page.

Styling and layout:

For responsiveness and usability, I considered three different sizes:

- 1- Up to 768px : Small tablets and phones
- 2- 768px to 1024px: Medium sized tablets
- 3- Larger than 1045px: Desktop screens

I used gemini AI generation to produce a LOGO relevant to the website topic[11]. In the css, I defined three css variables for colors so they can be applied throughout the stylesheet and give the project a consistent theme. In order to validate the HTML, check accessibility, and share the work with others, I deployed the web site by cloning the git repository into the Render and built an project there. The URL address for the created website is <https://internet-history-project.onrender.com>. I used this for HTML validation in <https://validator.w3.org> and the result was as following:

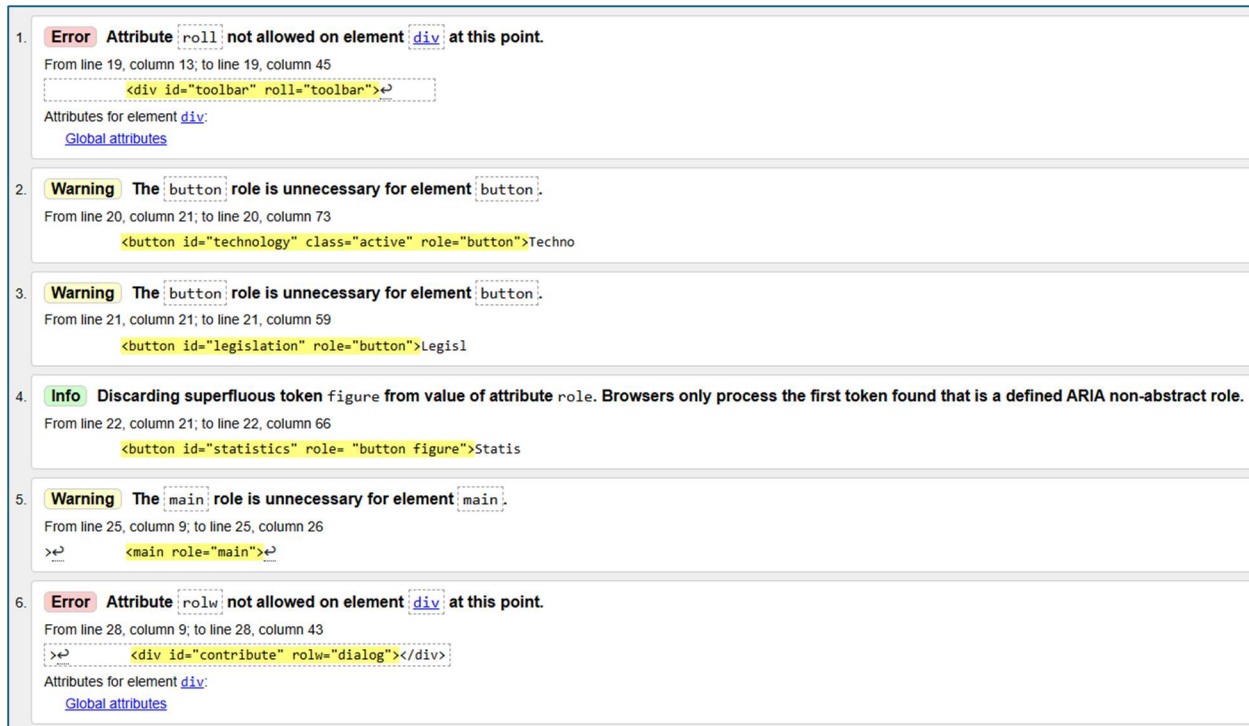


Image 2- First result from W3C Markup Validation Service

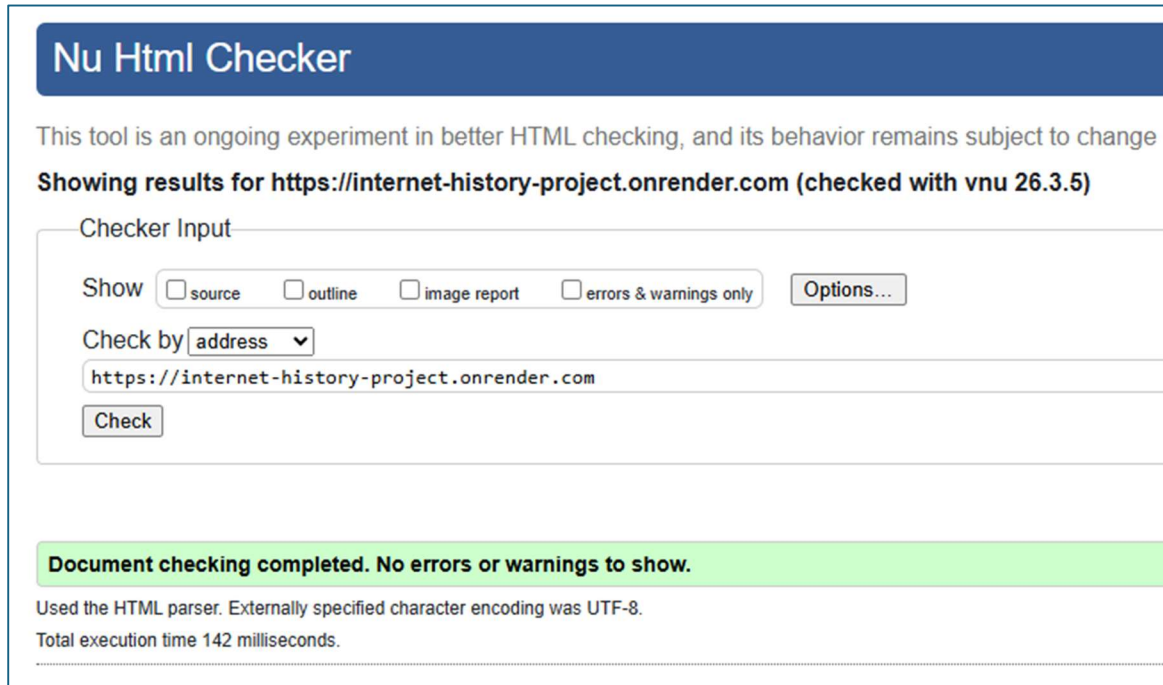
to address the errors I performed followings:

Item 1: Removed `roll="toolbar"` from `<div>` and added `role="navigation"` and `aria-label="Navigation between pages"` to it [10].

Item 2 to Item 5: Removed `role="button"`

Item 6: Removed `rolw="dialog"` and added `role="form"` and `aria-label="Navigation between pages"` to the `<div>`

The result after modification was as following:



Nu Html Checker

This tool is an ongoing experiment in better HTML checking, and its behavior remains subject to change

Showing results for <https://internet-history-project.onrender.com> (checked with vnu 26.3.5)

Checker Input

Show ☐ source ☐ outline ☐ image report ☐ errors & warnings only [Options...](#)

Check by [address](#) ▼

<https://internet-history-project.onrender.com>

[Check](#)

Document checking completed. No errors or warnings to show.

Used the HTML parser. Externally specified character encoding was UTF-8.
Total execution time 142 milliseconds.

Image 3- Second result from W3C Markup Validation Service

I also improved the accessibility of the website pages by applying role and aria-label attributes to non-semantic elements and obtained the final accessibility check results from AChecker as follows:

Congratulations! No known problems.

```
<a target="_blank" href="https://achecks.org/achecker#wcag2aa,https://internet-history-project.onrender.com/"></a>
```



- ✓ Does your content change frequently?
- ✓ Does your website have too many pages to keep up with?

Upgrade to have ACHECKS monitor your entire domain and get reports regularly!

[SEE THE SUBSCRIPTION OPTIONS](#)

Image 4- Final result from W3C Markup Validation Service

References:

- [1] Statcan. "Web Data Service (WDS) User Guide". Internet: <https://www.statcan.gc.ca/en/developers/wds/user-guide>, 2025 [Accessed March 7th].
- [2]Mozilla. "429 Too Many Requests". Internet: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/429>, 2025 [Accessed March 7th].
- [3]James Hibbard. "Delay, Sleep, Pause & Wait in JavaScript". Internet: <https://www.sitepoint.com/delay-sleep-pause-wait/>, 2023 [Accessed March 7th].
- [4]unpkg. "vis-timelin". Internet: <https://app.unpkg.com/vis-timeline@8.5.0/files/README.md>, 2025 [Accessed March 7th].
- [5]Github. "vis-timeline". Internet: <https://github.com/visjs/vis-timeline>, 2026[Accessed march 7th].
- [6]unpkg. "Timeline documentation". Internet: <https://unpkg.com/vis@0.5.1/docs/timeline.html>, [Accessed March 7th].
- [7]Mozilla. "HTML data-* global attribute". Internet: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Global_attributes/data-*, 2025 [Accessed March 7th].
- [8]Chart.js. "Getting Started". Internet: <https://www.chartjs.org/docs/master/getting-started/>, 2025 [Accessed March 7th].
- [9]Divine Hycenth. "Convert a Base64 data into an Image in Node.js". Internet: <https://dev.to/dnature/convert-a-base64-data-into-an-image-in-node-js-3f88>, 2020 [Accessed March 7th].
- [10]CanAdapt. "What happens with aria-labelledby, aria-label and aria-describedby". Internet: <https://www.davidmacd.com/blog/does-aria-label-override-static-text.html>, [Accessed March 7th].
- [11]Gemin, "Website Logo Generation: Internet History, Interent: <https://gemini.google.com/share/49031dc9f5d9>, 2026 [Accessed March 7th]